FINAL REPORT
REPORT DUE: May 6, 2025

Group: 1

1 Introduction

Our project, Grade Gator, is a grading software system designed to streamline the organization of courses, assignments, and submissions through an intuitive and visually appealing interface. While it is built with a focus on computer science courses — allowing instructors to create coding assignments with test cases and providing automated feedback through an integrated autograder — the platform also supports the submission and grading of PDF documents, making it adaptable to a variety of disciplines. Our primary customer is Davidson College, specifically Dr. Lim and Dr. Heyer, and our intended users are professors and students across courses that would benefit from a more efficient grading workflow. For our development process, we followed an Agile-Scrum methodology, including defined roles such as Scrum Master and Product Owner, and conducted development in two-week sprints to ensure continuous progress and adaptability throughout the project.

2 Novelty

Grade Gator introduces several novel features that set it apart from existing grading software. Unlike competitors such as Gradescope and CodePost, Grade Gator is completely free and offers a more personalized, visually engaging experience through custom colors and branding. While we aimed to implement standards-based grading to better serve institutions like Davidson College, time constraints prevented full integration — though it remains a core goal. Our platform is designed to enhance both the student and instructor experience by including features such as rubric visibility prior to submission, automatic grade calculation, inline feedback comments on coding submissions, and required acknowledgment of feedback before viewing grades. These features aim to create a more transparent, interactive, and efficient grading process that existing solutions often lack.

3 Customers and Stakeholders

3.1 Primary, Secondary, and Any Other Stakeholders

1. The primary customers of our project were Dr. Lim, Dr. Heyer, and the Davidson Math & CS department, who approached us and pitched this project.
2. The secondary customers of our project were Davidson College as a whole and potentially students and faculty from other schools. They were our secondary customers because they were not directly involved in the development process of our project, but we created it with them in mind as users.

3.2 Meeting Log

Date	Customer	Description
February 12, 2025	Dr. Lim, Dr. Pulaj, Dr. Williams	The customers want a web-based assignment manager that includes code autograding functionality and allows them to provide assignments to their students, give feedback on said assignments, and organize student responses in an efficient and effective manner. The stakeholders have frustrations with the product they currently use to provide this functionality and want to create a platform to serve the same purpose but that is tailored to their needs and easier to use.
March 19, 2025	Dr. Lim, Dr. Williams, and Dr. Heyer	Dr. Williams requested the ability to upload rubrics and have standards based grading. Dr. Lim emphasized that he would like to see a usable front end as soon as possible. Dr. Heyer requested support for PDF assignments and multiple choice quizzes. Our customers asked for visibility of the rubric to students and a manual GUI interface to create and grade assignments. They also expressed interest in the option to select which parts of the rubric are available to students. The customer clarified their focus on working software over well-designed software.
April 4, 2025	Dr. Lim and Dr. Pulaj	Our customers suggested focusing on adding the functionality of submitting multiple files at once as well as creating more documentation for future developers to work off of. We also needed to work with T&I and our group to hopefully gain

		access to an organization-level AWS account during the next sprint.
April 25, 2025	Dr. Lim	We met with the customer to discuss the future of our project and how to transfer our work to an organizational AWS account so that future developers can easily work on it. Our customer requested that we transfer our work to the new account when available and create documentation for everything we completed so far.

4 Product Backlogs

4.1 Past Iteration Product Backlogs

Iteration Period	List of Backlog Items
April 3rd - April 29th	- Create a gradebook that supports standards based grading - Connect the front end to the autograder - Develop documentation and to-do list for next set of developers - Connect creating assignments on front end to back end database - Enable the submission and grading of assignments with multiple files - Create PDF submission support for assignments - Host the frontend on a website rather than a local host. - Organize the backend databases so necessary information is stored and can be easily accessed. - Require login authentication. - Develop standards based grading ability - Expand GUI for instructors to create assignment rubric - Research and implement ways to securely store student data once on a shared server. - Research the potential for Moodle integration of class rosters. - Allow students to view rubrics for assignments. - Expand the types of assignments from coding assignments to other file submissions and multiple choice/short response quizzes. - Meet with professors in the math department to discuss needs of department assignments. - Add fun gator graphics

4.2 Product Backlog for the Next Iteration

- Transfer project from personal AWS and GitHub accounts to the GradeGator admin account
- Host the site on a server
- Integrate Davidson SSO
- Enable token authentication
- Create standards based grading functionality
- Complete implementation of grading for PDF submission
- Create gradebook functionality
- Expand GUI for instructors to create assignments
- Include review of instructor comments and grading on the student view of assignments.
- Expand the types of assignments from coding assignments to other file submissions and multiple choice/short response quizzes.

Assuming you continue the project, list the product backlog items built from the last sprint review with the customers.

5 Sprint Backlogs

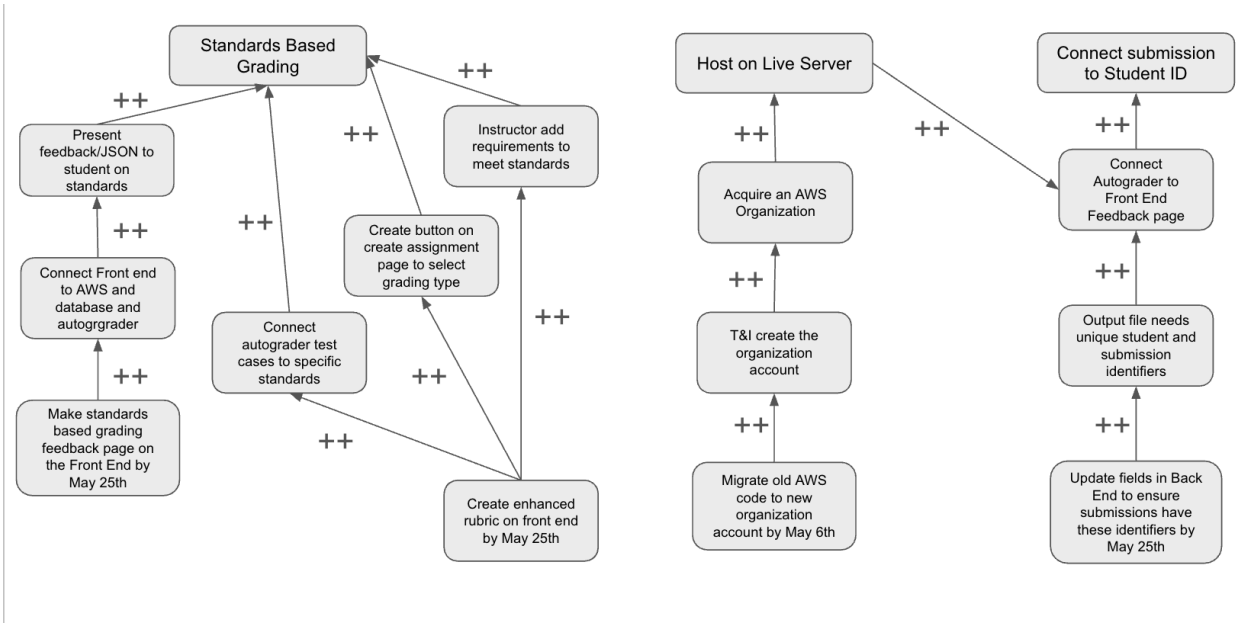
4.1 Past Iteration Sprint Backlogs

Iteration Period	List of Backlog Items
April 3rd - April 29th	- Connect the front end to the database and then to the autograder - Enable PDF submission and autograding - Set up an organizational AWS account and transfer everything over - Enable the submission of multiple files - Host the frontend on a server - Create documentation of our system for the next team of developers

4.2 Sprint Backlog for the Next Iteration

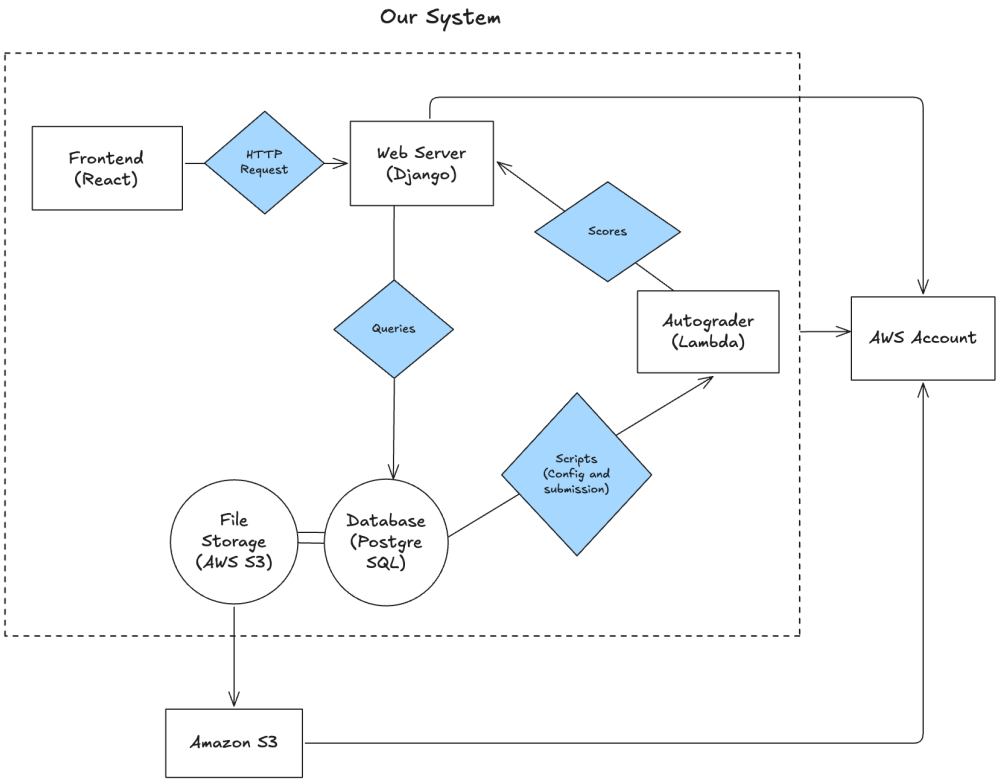
- Transfer project from personal AWS and GitHub accounts to the GradeGator admin account
- Host the site on a server
- Complete implementation of grading for PDF submission
- Expand GUI for instructors to create assignments
- Include review of instructor comments and grading on the student view of assignments.
- Expand the types of assignments from coding assignments to other file submissions and multiple choice/short response quizzes.

6 SMART Goals



7 System Description

7.1 Block Diagram

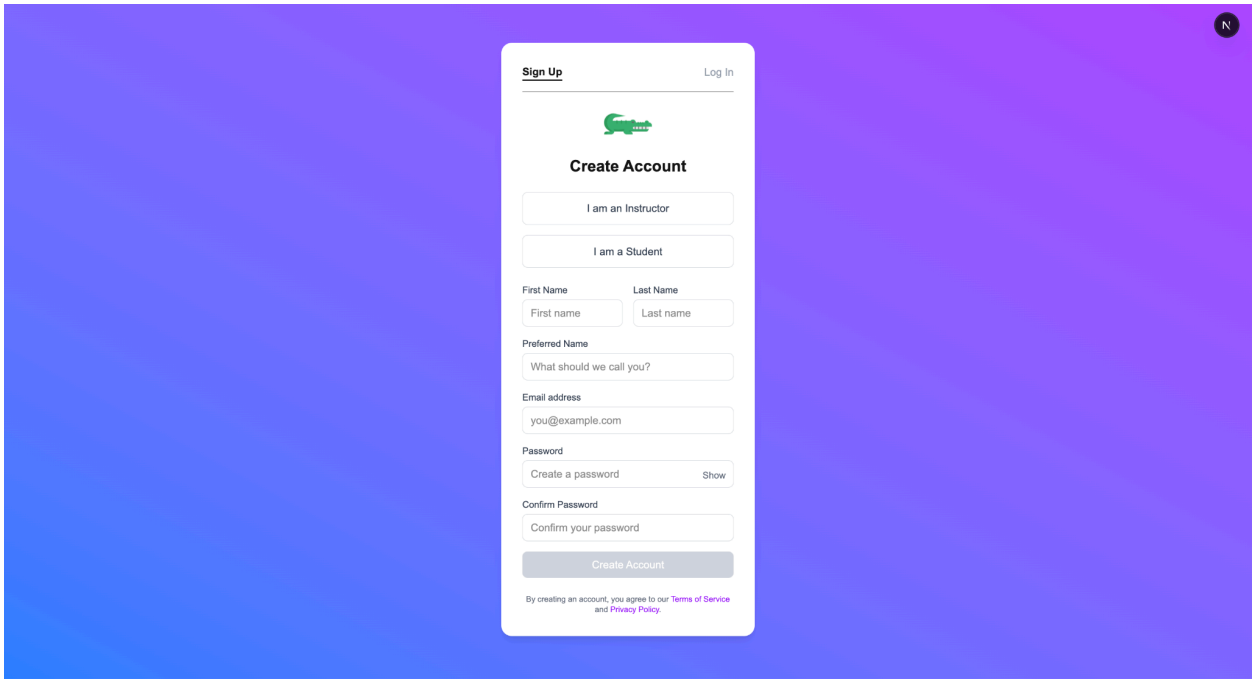


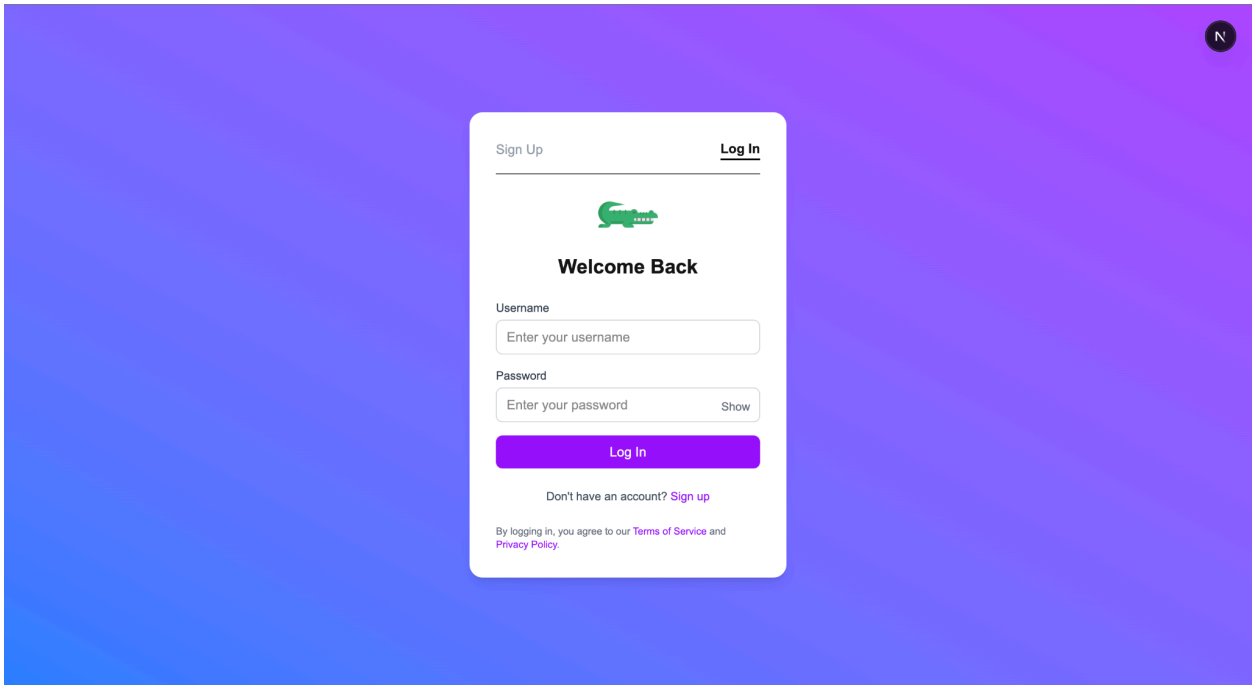
8 Current Status

8.1 Screenshots and Explanations

Front End

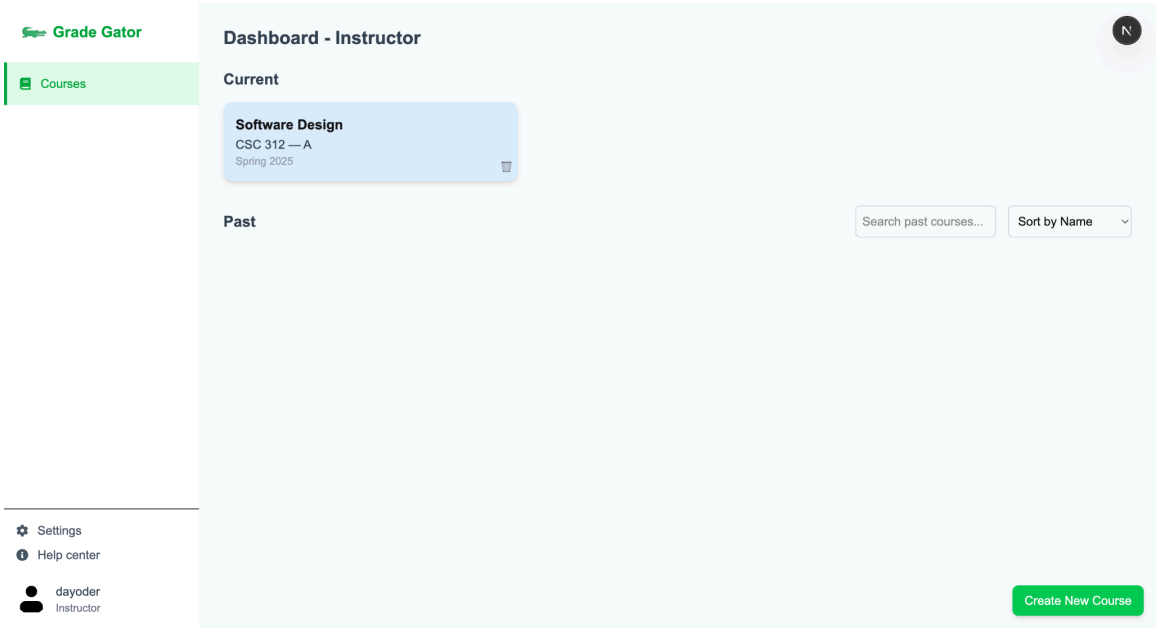
Currently, the sign up and log in functions are both connected to the backend. The user can register as an instructor / student and login with their username.

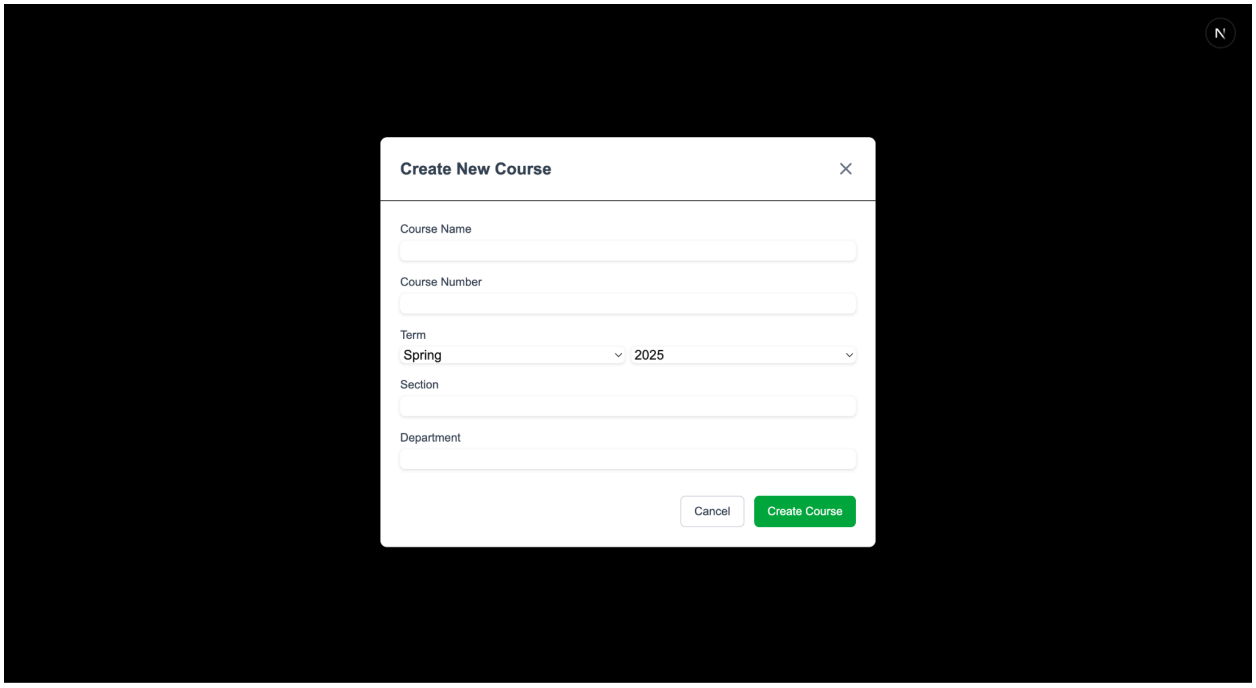




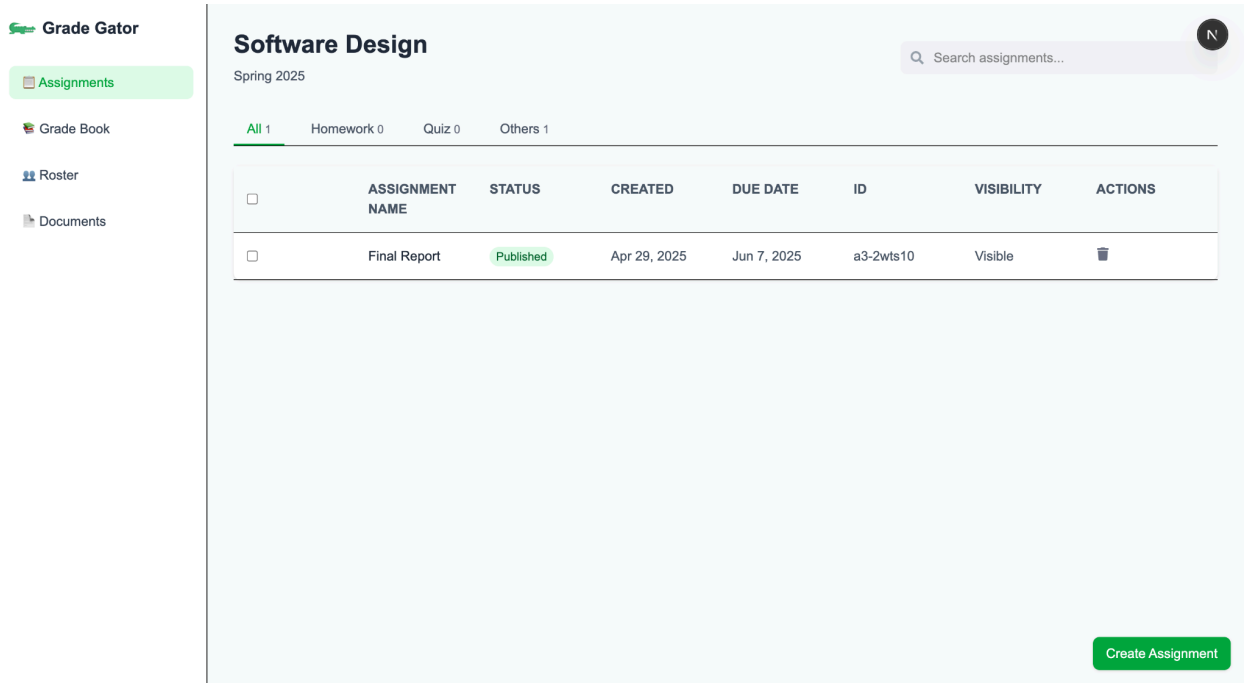
Instructor View

Instructors can see all the courses they've created in the current and past sections. They can delete courses if they'd like. They can create new courses by clicking on the "Create New Course" button.

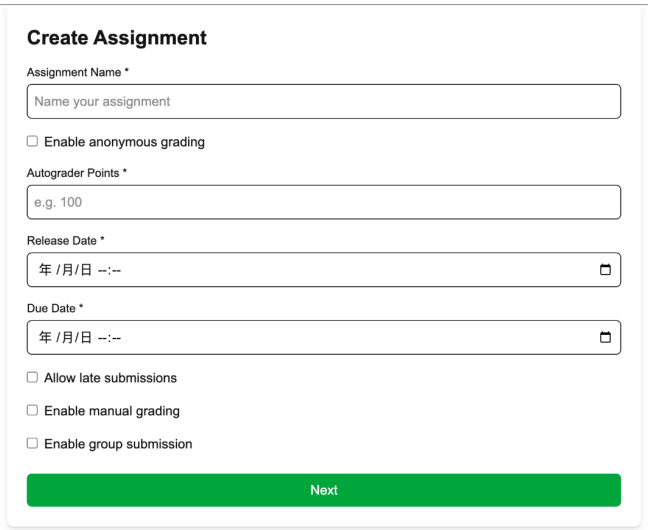


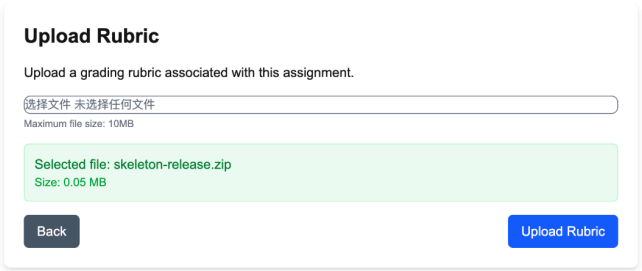


After clicking into certain courses, they can see the assignment page. When clicking the Create Assignment button, they can create new assignments.



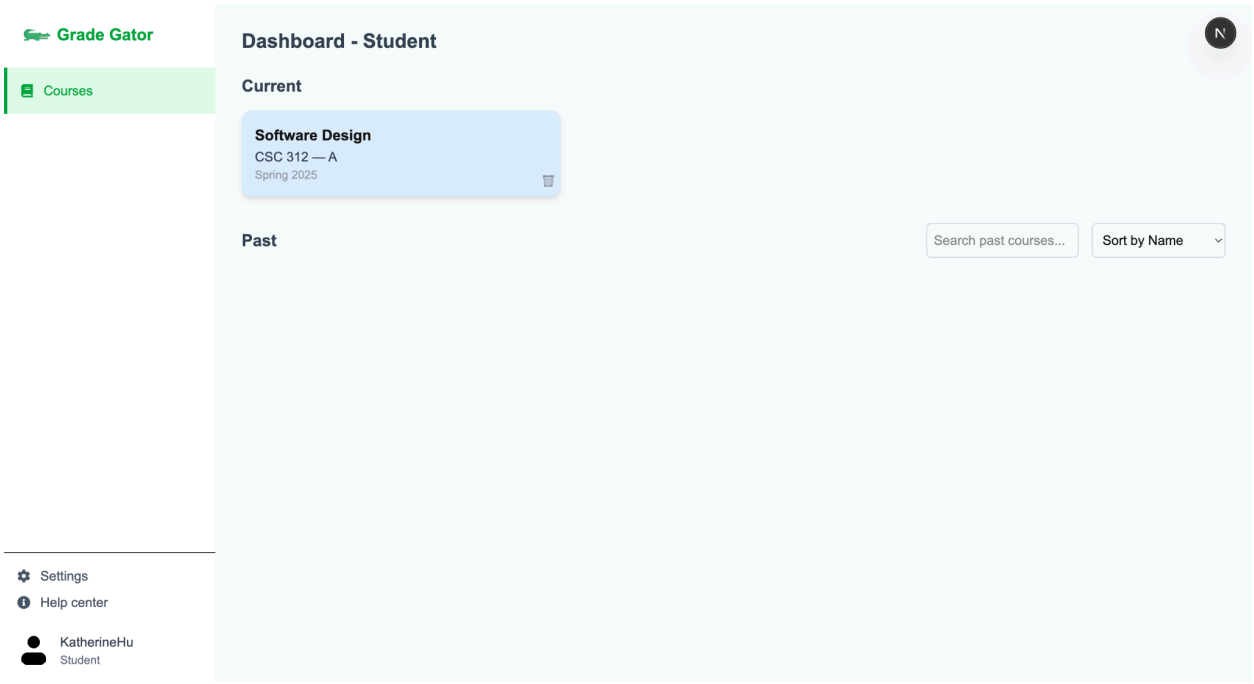
The instructor needs to specify assignment names, points, release date, due date, and etc. when creating an assignment. They also need to upload a ZIP file containing all the files required for autograding. After hitting the upload button, the submitted ZIP file will be stored in Amazon S3.



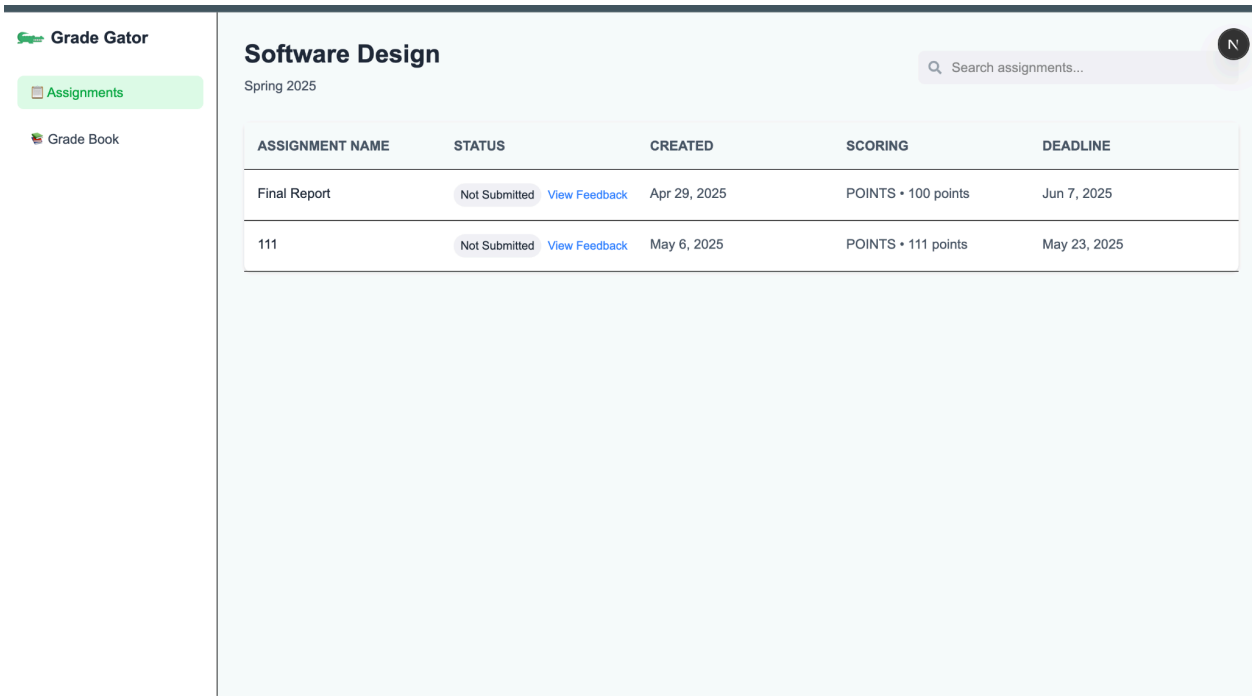


Student View

The student's view is very similar to the instructor's view except that a student can’t create any courses.

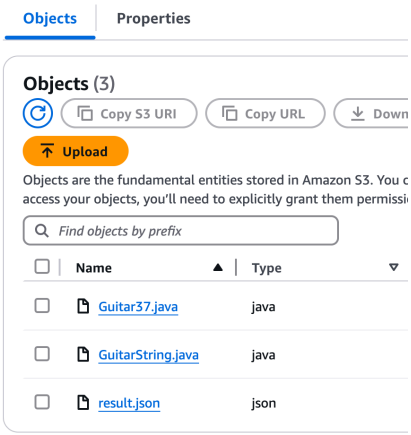


When clicking into the student view, it shows the title of the course, the semester, the search bar, and the assignments. When clicking on one of the assignments, it allows the student to upload their files.

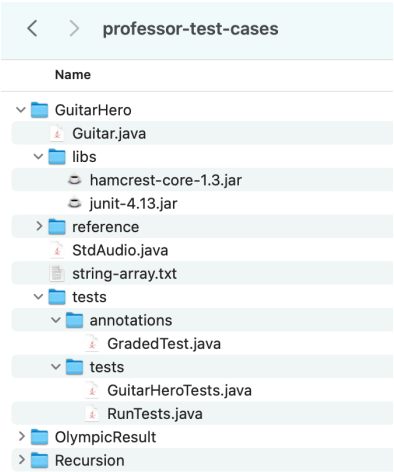


Autograder Lambda Functions

GuitarHero_1745950231701/




```
{"status": "success", "output": "Tests run: 10
Tests failed: 0
{
  \"passed\": true,
  \"total\": 10,
  \"failed\": 0,
  \"testResults\": [
    {
      \"testName\": \"testSmallBuffer\",
      \"passed\": true,
      \"input\": \"frequency = 44100.0\",
      \"expectedOutput\": \"throws IllegalArgumentException\",
      \"actualOutput\": \"throws IllegalArgumentException\"
    },
    {
      \"testName\": \"testFreqArraySize\",
      \"passed\": true,
      \"input\": \"array = [1]\",
      \"expectedOutput\": \"throws IllegalArgumentException\",
      \"actualOutput\": \"throws IllegalArgumentException\"
    },
    {
      \"testName\": \"testWithFrequency\",
      \"passed\": true,
      \"input\": \"frequency = 10000.0\",
      \"expectedOutput\": \"All samples should be 0.0\",
      \"actualOutput\": \"All samples were 0.0\"
    },
    {
      \"testName\": \"testTic\",
      \"passed\": true,
      \"input\": \"array = {0.0, 0.25}, tic() twice\",
      \"expectedOutput\": \"sample() = 0.1245\",
      \"actualOutput\": \"sample() = 0.1245\"
    },
    {
      \"testName\": \"testPluck\",
      \"passed\": true,
      \"input\": \"frequency = 10000.0, then pluck()\",
      \"expectedOutput\": \"Samples between -0.5 and 0.5, not all 0\",
      \"actualOutput\": \"Samples were in range and not all 0\"
    },
```

As for the autograder, its functionality remains the same, yet introduces some new features. A lambda function to manage .pdf submissions has been implemented, which is very simple in theory; just sending the submitted pdf to the graded-results bucket. The other feature is added to the original JavaSubmissionGrader function that now accepts submissions for assignments that require multiple files for that submission. The student must upload the two files they are submitting as a .zip file, and they are grouped together in the graded-results bucket. The result.json format works accordingly as well. Changes were made to the professor-test-cases bucket, the file organization has changed slightly, but each test case for each current working assignment (Recursion, OlympicResult, GuitarHero) has been updated.

8.2 Software Testing

8.2.1 Test Cases

1) Grade creation and representation: inside grading/tests.py. It tests the grade model creation and string representation, ensuring that each posted grade can be properly associated with submission and instructor. It is critical for tracking the grade and feedback.
2) Assignment with Student Accommodations: inside assignments/tests.py. It tests whether the program allows customizing accommodations for students (e.g. extra time). It verifies the support for accessibility requirements and tests linking accommodations to specific students or assignments. It is critical since assignment accommodations are crucial for students who need them.
3) Course creation and ordering: inside courses/tests.py. It tests whether the course model can by default order the courses by course numbers. It ensures that all courses are sorted logically in UI listings, which is essential for user experience.
4) Admin user creation with roles: inside accounts/tests.py. It tests the admin interface for creating users with roles (e.g. is_student). It validates role-based control setup, which is critical for security, ensuring that admins can properly configure user permissions.
5) Student-Course Enrollment Relationship: inside courses/tests.py. It tests the many to many relationship between students and courses. It validates that students can enroll in multiple courses and vice versa. It ensures the bidirectional queries can work, which is fundamental for the gradebook and dashboards.

8.2.2 Combinatorial Testing: Define Factors

Factor: User Type. Values: Student, Instructor, Admin, Unauthenticated User.

Factor: Submission Status. Values: Submitted, Not Submitted, Late, Resubmitted.

Factor: Grade Status. Values: Not Graded, Graded.

Factor: Course Types. Values: Required, Elective, Lab.

Factor: User Access Permissions. Values: Read-Only, Edit Access, Full Access, No Access.

9 Project Management

Date	Description
	No big changes this sprint

10 Sprint Review and Retrospective

10.1 Sprint Review

The key focus of this sprint was to continue connecting the frontend and backend to the autograder and streamlining the user workflow. Our customer also requested that the autograder be able to accept multiple file submissions, since multiple files are required for Data Structures homework assignment. Our primary goal was for an instructor to be able to create an assignment and send the rubric to AWS to be used for autograding. Then a student could see the assignment, submit their files, the files would be sent to AWS for autograding based on the instructor's rubric, and then the results of the autograder would be displayed in the student's assignment submission page. We were able to connect instructors' submissions of rubrics to the AWS bucket for the unique assignment it belongs to. We were able to complete the connectivity from the frontend to the AWS bucket via the backend, but we were unable to display the results of the autograder in the front-end. This limitation was due to operating under a personal AWS account rather than a AWS Organization Account through Davidson T&I. We worked with T&I to gain access to an organization account, but did not receive the access until the end of the sprint. Although the autograder is unable to send information to the front-end, the results of the autograder are displayed in an AWS submissions bucket. These results report if the test cases ran successfully.

The scope of the project has been cut back since its original inception. We were unable to implement core features of GradeGator, but through talking with customers, they remain integral to the project. Specifically standards-based grading is a key desire expressed by customers. We added space for a gradebook in our UX design and we envision the gradebook will allow instructors to specify if their course uses standards based grading. Then the instructor could set the parameters for the grading rubric, such as U-unsatisfactory, N-needs improvement, S-satisfactory, etc. Another core feature is the concept of in-line feedback for coding assignments. We also hard-coded an example of this feature in our UX, but ran out of time to implement it, mainly because we were unable to send autograder feedback back to the front-end, as mentioned earlier.

Throughout the project, our customers have reported satisfaction with our progress. In this sprint review, Dr. Lim expressed contentment with what we were able to accomplish while acknowledging there was much more to be expanded upon. One thing our group has enjoyed doing throughout the semester is imaging a complete UX design that integrated our gator mascot. Although we did not spend time on this integration (it has always been in the “Won’t have” of MoSCoW) we shared some of our ideas with the class. Namely, we envision that the completed GradeGator software will show a gator GIF walking across the screen to denote the running of the autograder rather than the spinning/loading symbol. Dr. Lim seemed very amused by this idea and our overall Gator branding. Our customers' key concern for the future is having accurate and descriptive documentation, which we focused on at the end of this sprint.

10.2 Sprint Retrospective

Our team continues to communicate well and split the work into subcategories that can be shared by one to three group members. We were very successful in making improvements to the autograder despite our setbacks with the AWS Organization account. However, a handful of issues came up that resulted from inconsistencies in the code from previous iterations. Our test suite was not yet thorough enough to catch them. Most of them were related to database schema changes and the corresponding migrations that had to be made. Ultimately the goals that were not met were due to time constraints and difficulty coordinating between our team and Davidson T&I. We could have begun meeting with T&I in a previous sprint, but we only realized the urgency of our need for an AWS Organization account once we began the current sprint. Our past issue of coordinating times to meet remained as a challenge. However, all of our team members were dedicated to finding times to meet, whether that was eating dinner while we did it, or staying together until 11pm, so the challenge was more of an inconvenience than a hindrance to our progress as a group.